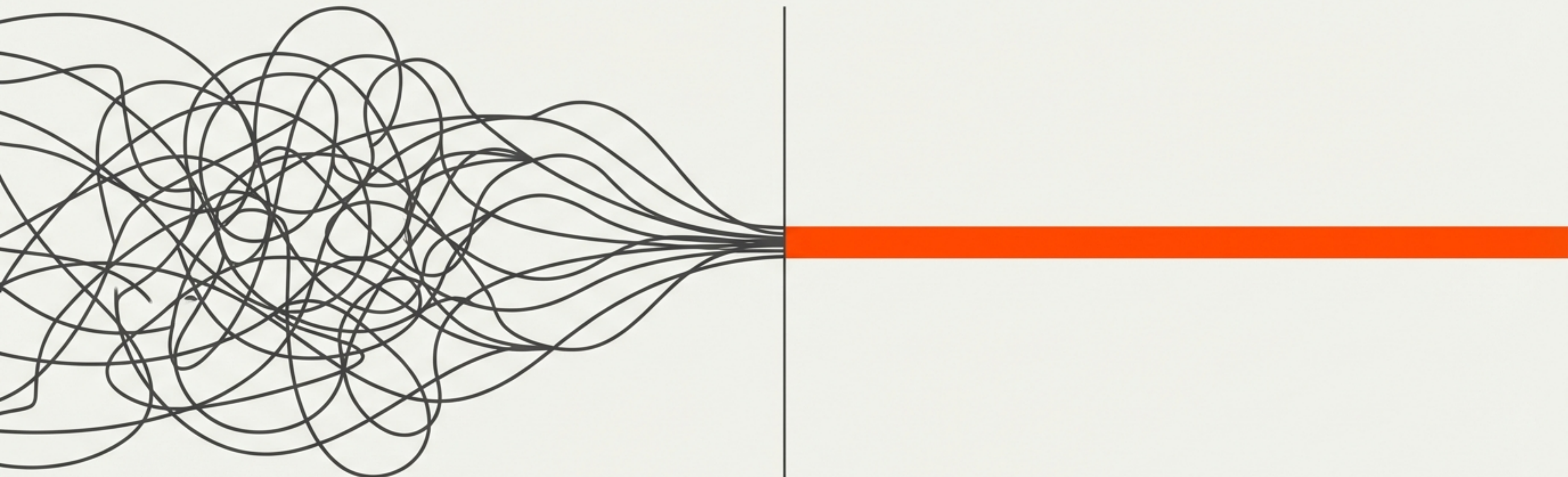




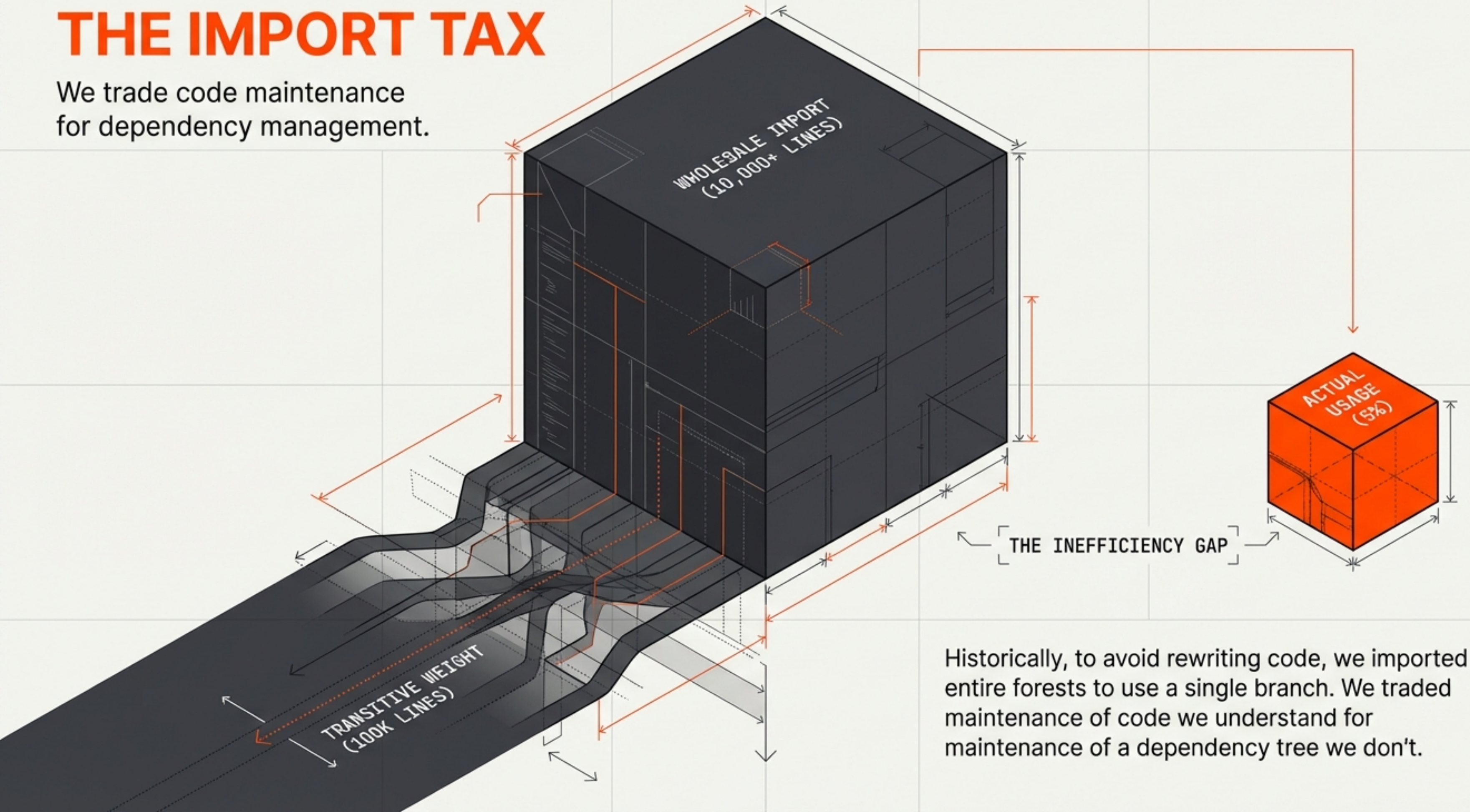
Dependency Adaptation

The Economic Shift from Import to Copy.

Leveraging LLMs to transform dependency management, reduce attack surfaces, and reclaim code agency.

THE IMPORT TAX

We trade code maintenance
for dependency management.

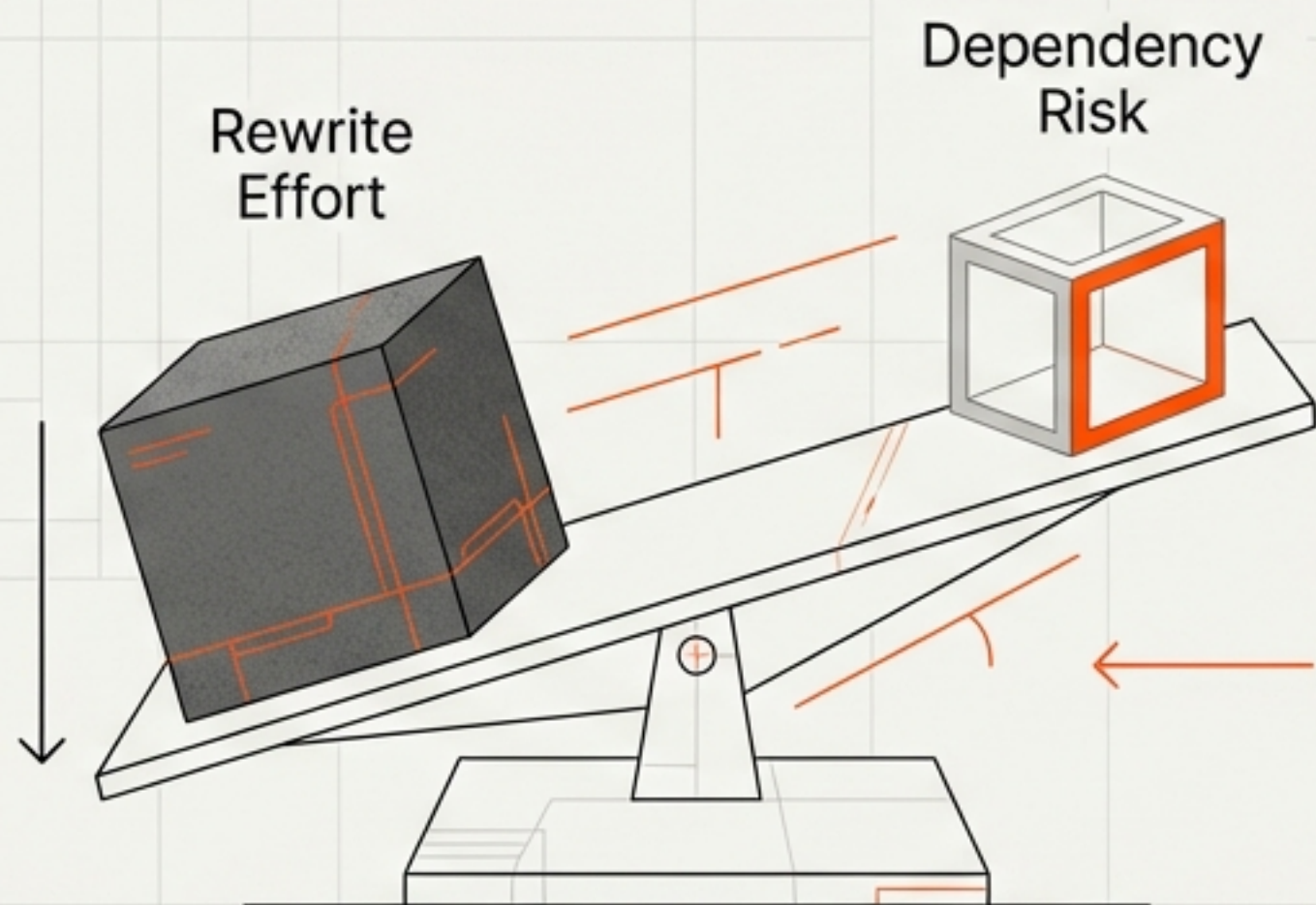


Historically, to avoid rewriting code, we imported entire forests to use a single branch. We traded maintenance of code we understand for maintenance of a dependency tree we don't.

The Economic Inversion

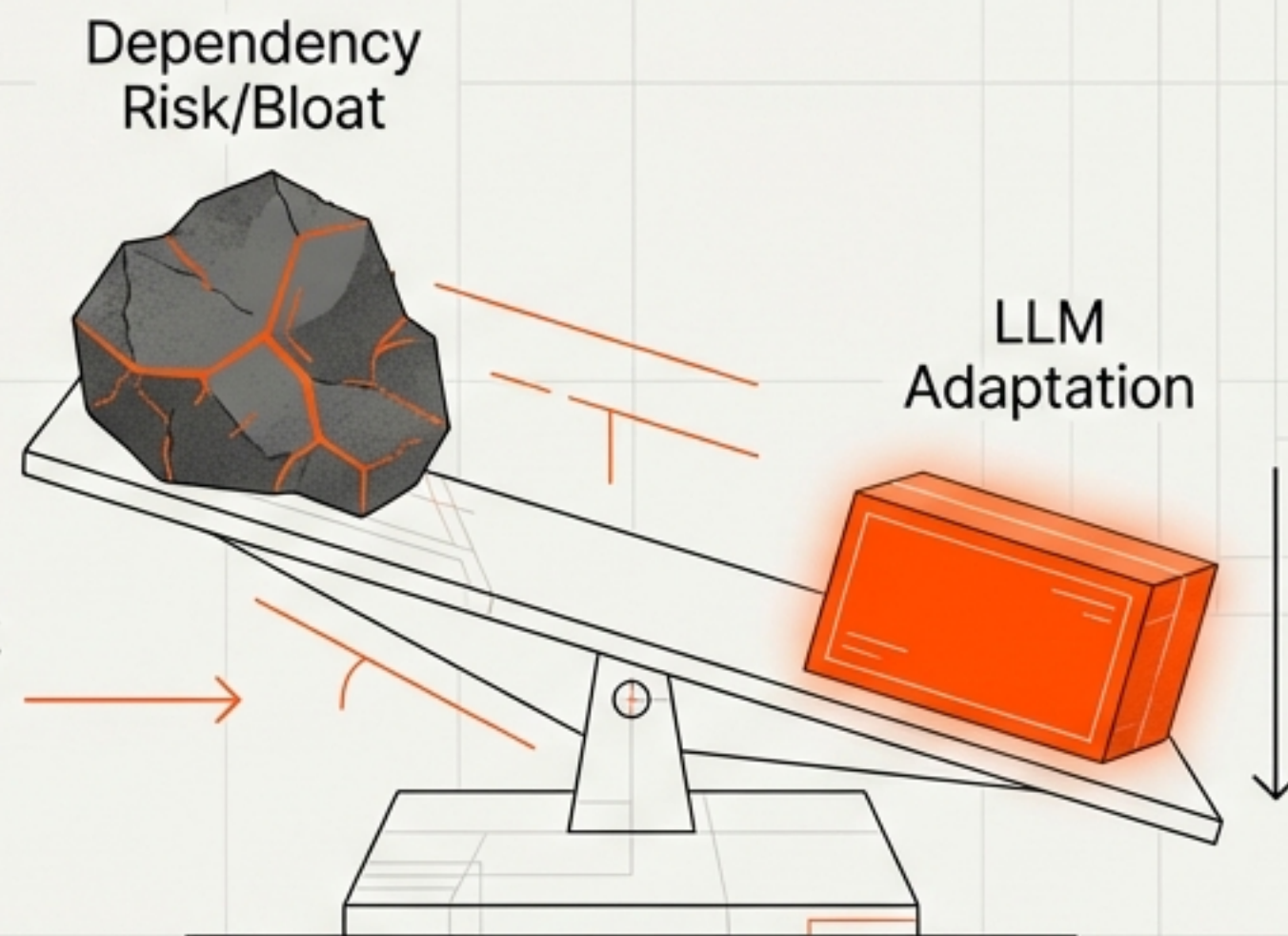
Pre-LLM Economics

Rewrite Cost > Import Tax



Post-LLM Economics

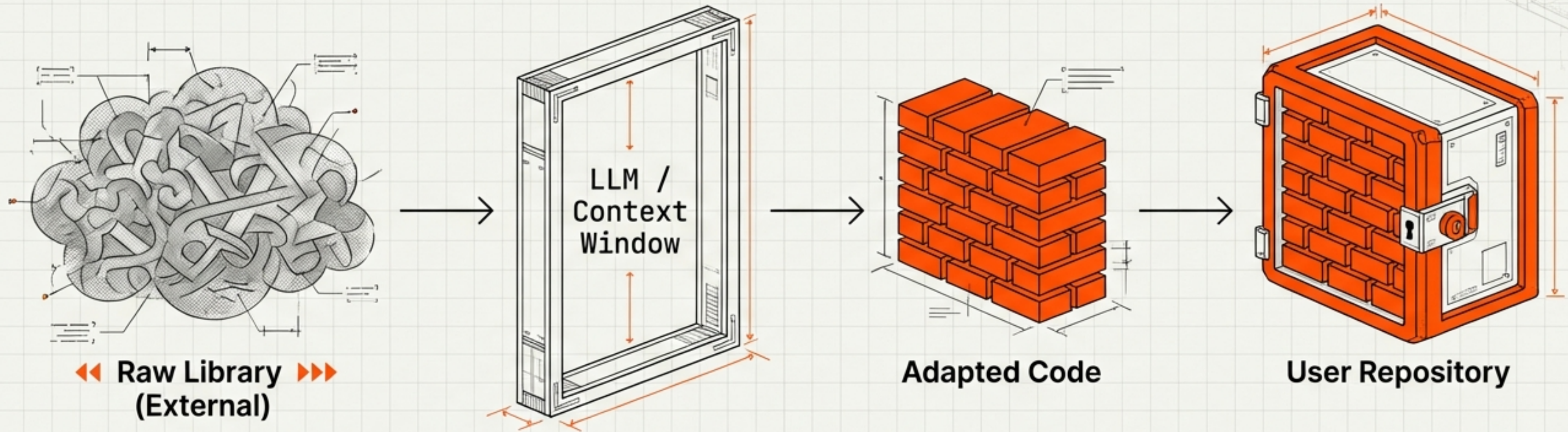
Import Tax > Adaptation Cost



Catalyst: The cost of transformation has collapsed.

Strategic Copy over Wholesale Import

Dependency Transformation, not Elimination.



From

`npm install` (Import everything, use 5%)

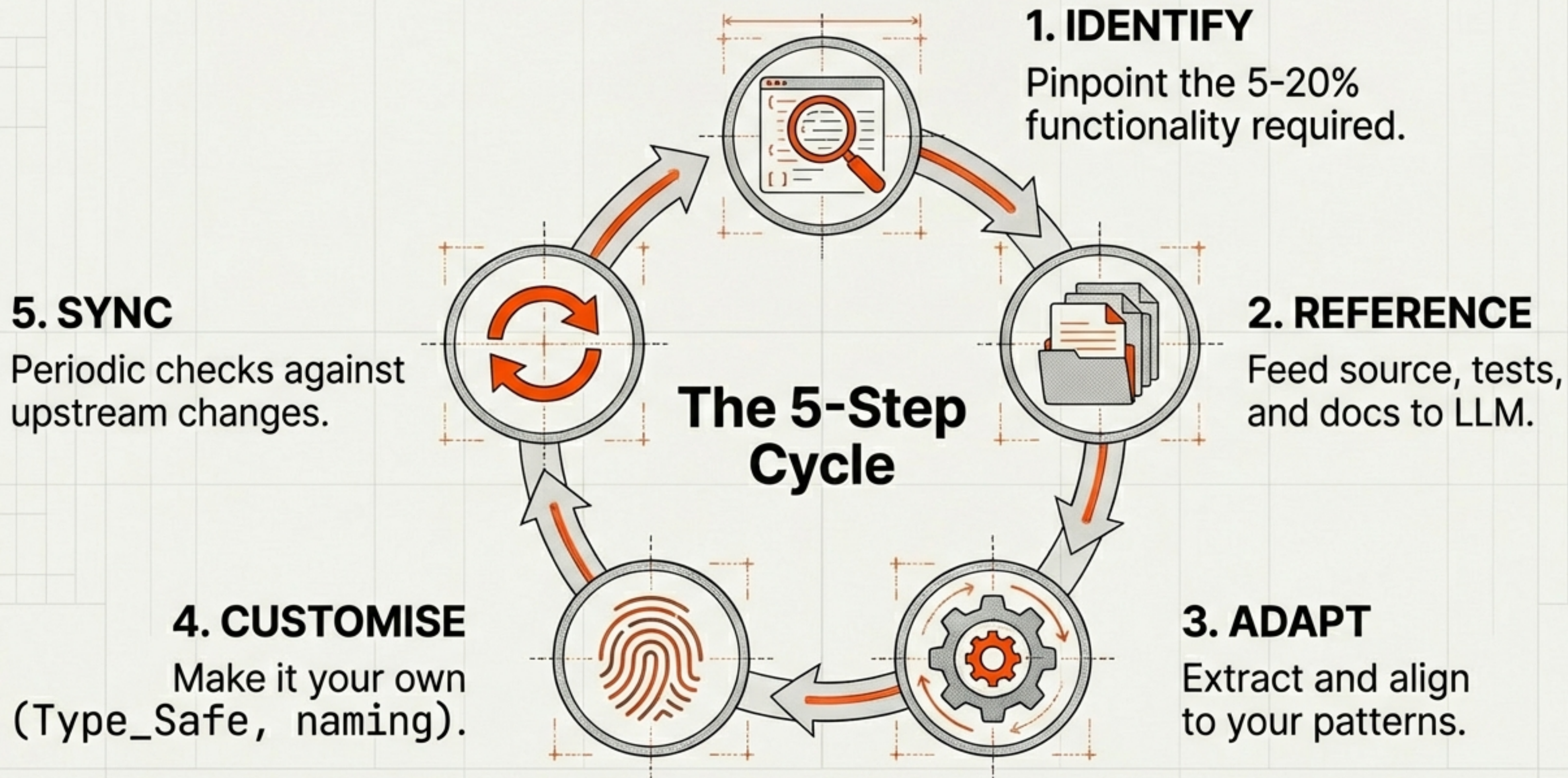
To

Strategic Adaptation (Copy 5%, Customise, Maintain)

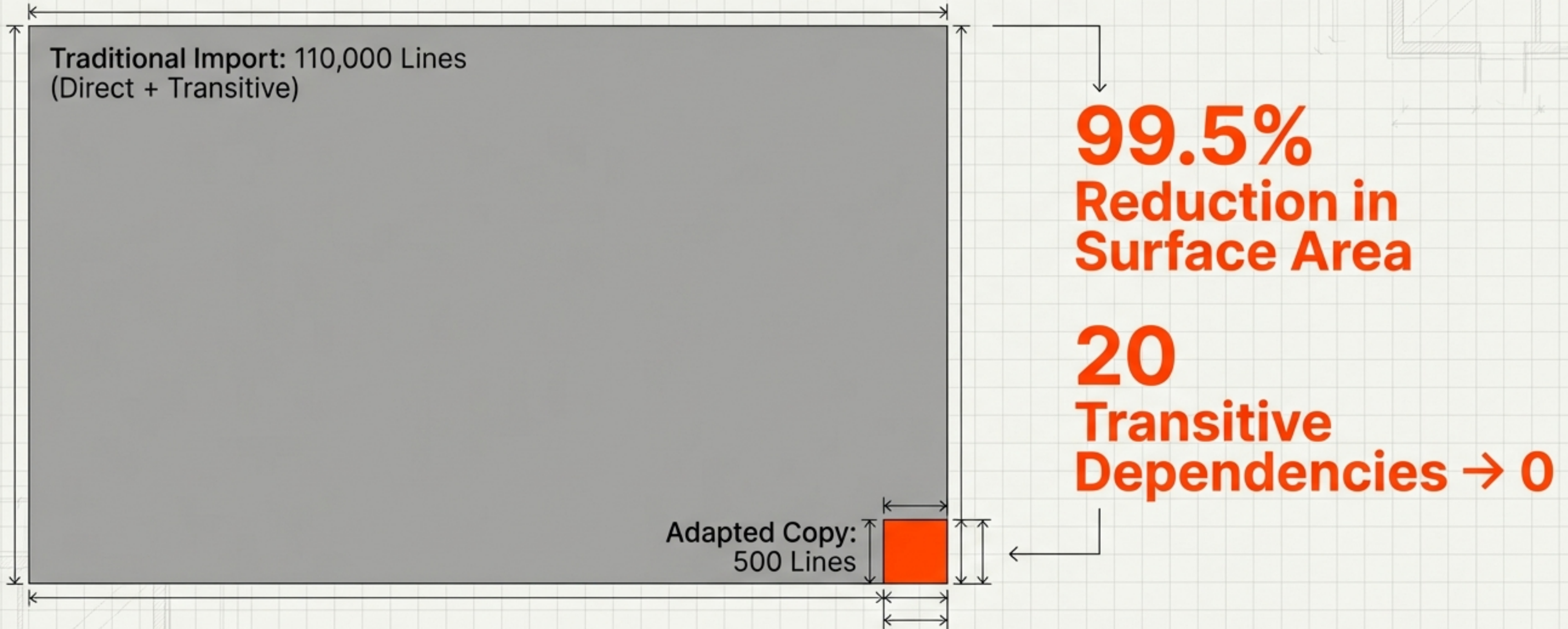
Result

Type_Safe, Schema-Aligned, Owned Code.

The Adaptation Lifecycle



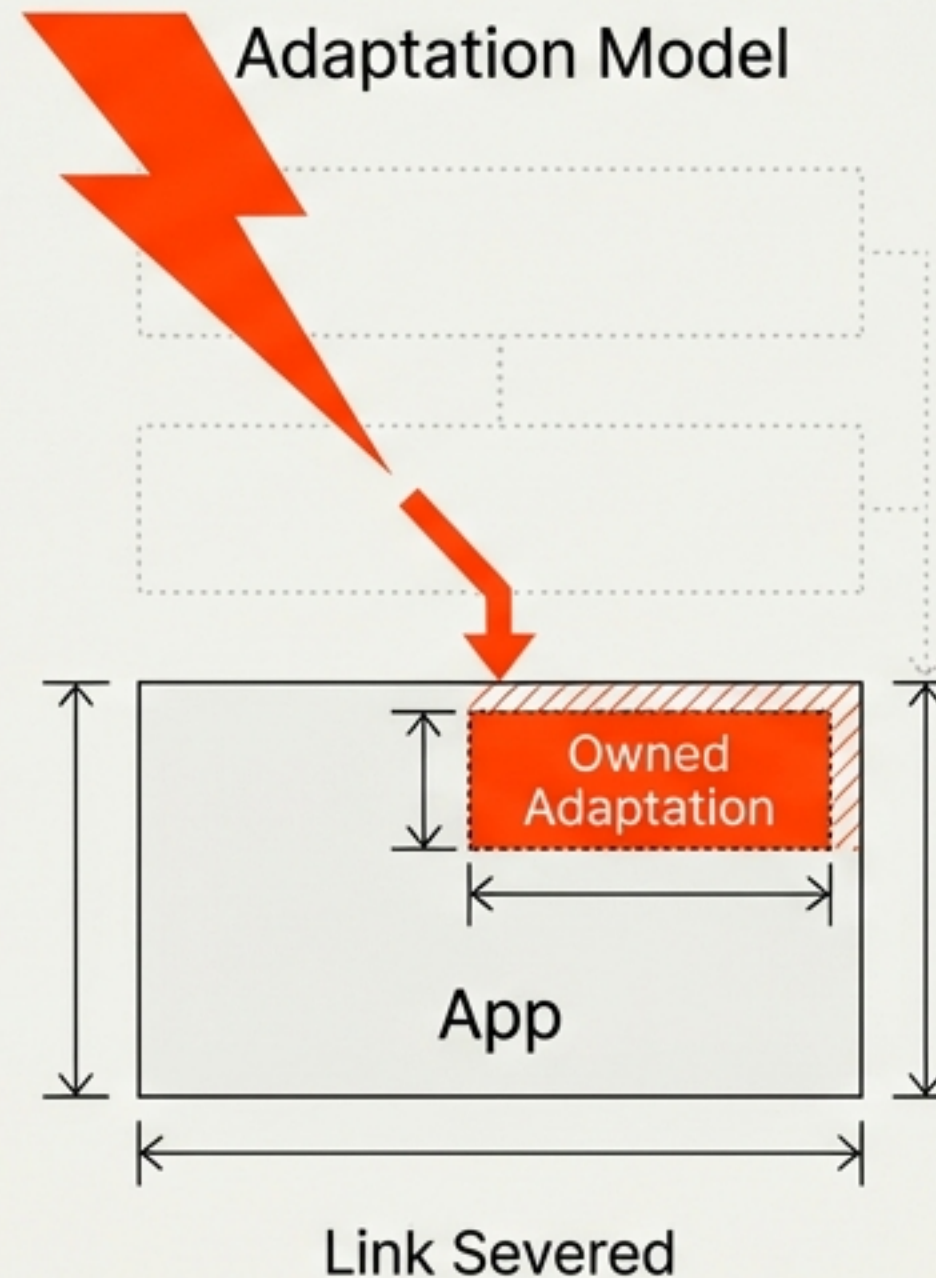
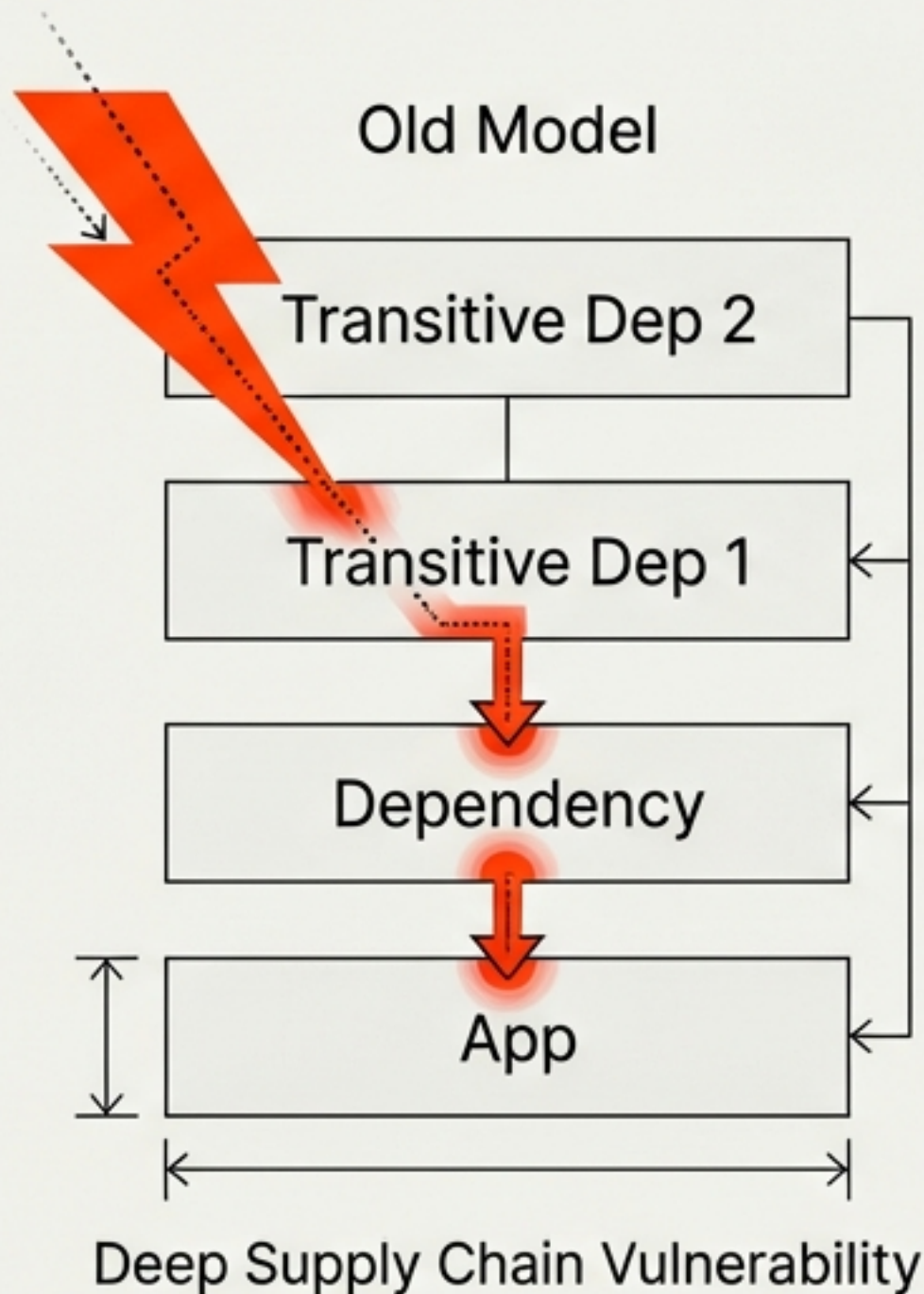
The Mathematictics of Surface Area



"You were just doing it for 100,000 lines of code you didn't understand instead of 500 lines you do."

The Security Dividend

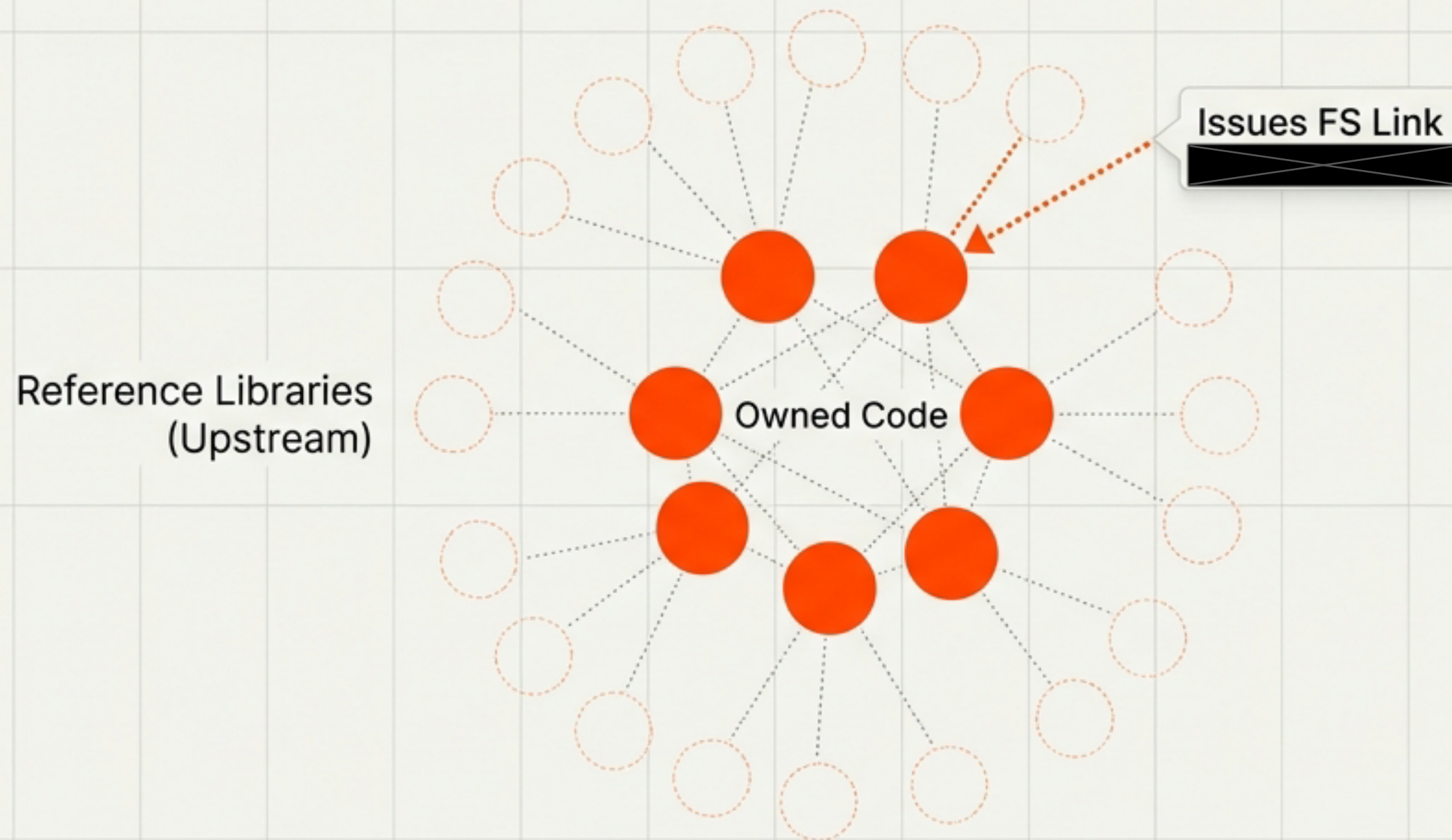
Drastically Reducing the Attack Surface.



- **Zero** Transitive Surprises
- **Auditable Surface Area**
- Shift from “Does CVE-2024-123 affect us?” to “Do we have this function?”

Code Agency & The Graph

Moving from Opaque to Transparent.



Concept: Issues FS

Treat adapted dependencies as nodes in a graph. Link back to source version, commit, and specific functions.

Agency Score:
Increasing from
10% Owned to
70% Owned.

Decision Framework: When to Adapt?

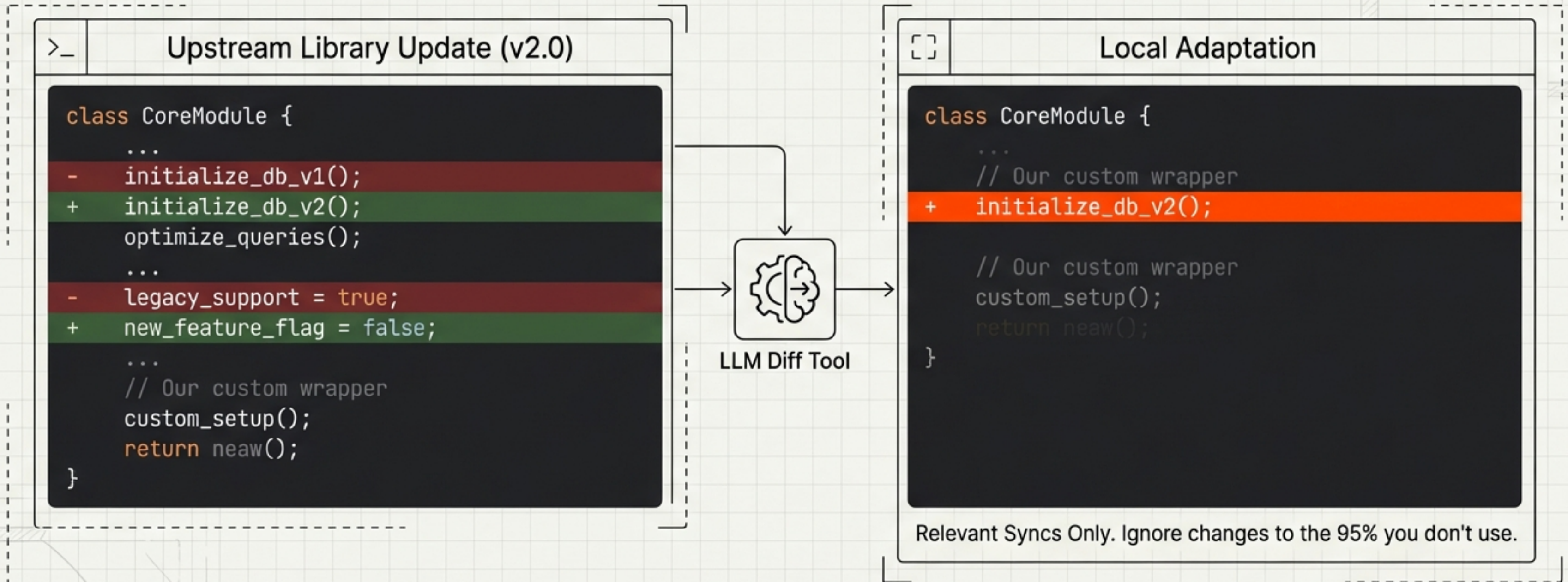


Rule:

- Adaptation is for the bulk 'glue code' and utilities, not deep domain cores.

The Maintenance Rebuttal

Automated diffing makes ownership viable.



Objection: "Now I have to maintain it."

Reality: You always had to. Now you have tools to filter the noise.

A New Commercial Model

The Adaptation Service for Open Source Maintainers.

Traditional Support

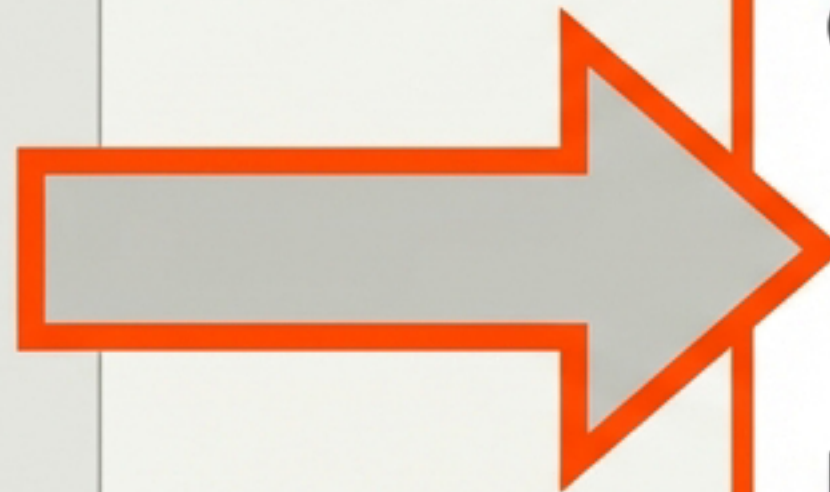
Customer gets:

- Generic help, full library bloat.

Maintainer gets:

- Hourly support work.

High Value
Shift



Customised Adaptation

Customer gets:

- Minimized, secure, **tailored code injection. No transitive tree.**

Maintainer gets:

- Scalable revenue, leveraged domain expertise.

The Delivery Model

Maintainer / Curator

Creates Adaptation

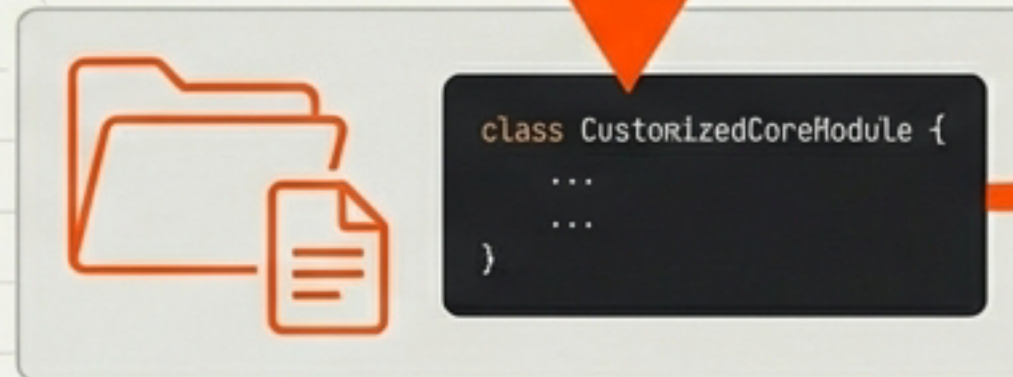
Delivery Channel

Private Injection

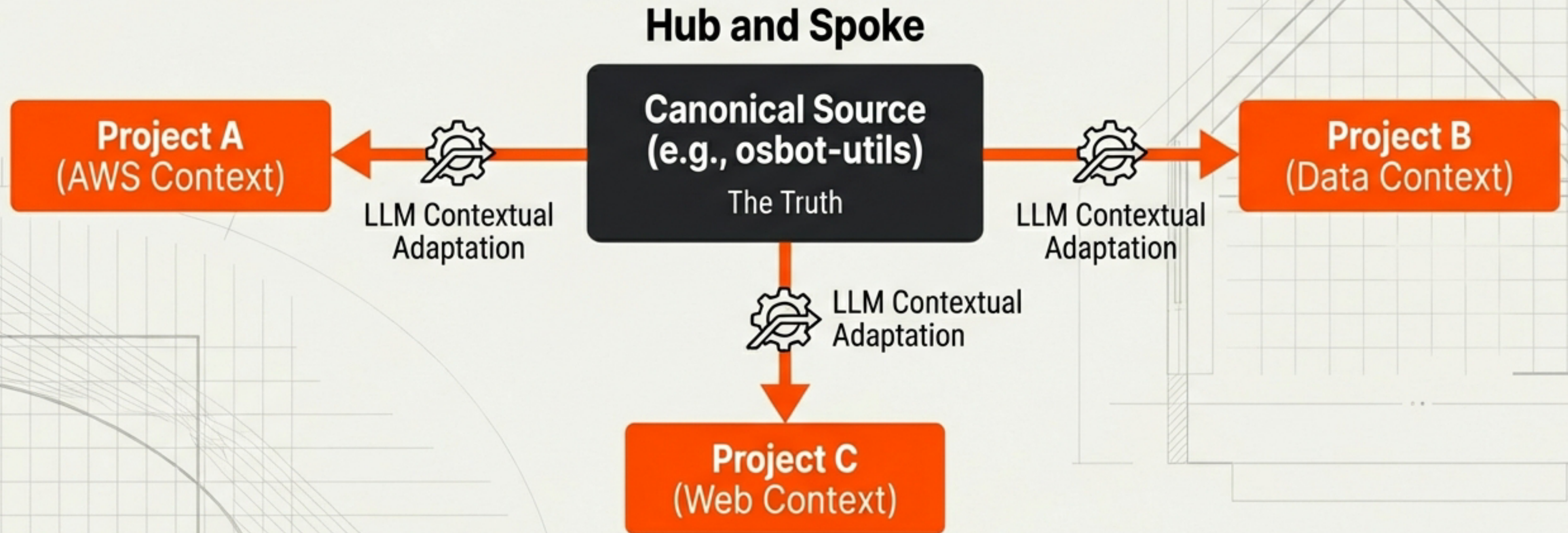
Customer Repo

1. **Relevant Syncs**
(Upstream Changes)
2. **Vulnerability Response**
(Tailored Patches)

Subscription



Reference Implementation



The author's workflow: Maintain a central personal library, but inject only adapted snippets into specific projects. High velocity, low bloat.

Strategic Roadmap

Architect



Define Adapt vs. Import criteria.
Build the decision matrix.

Developer



Execute first adaptation.
Measure line count reduction.

AppSec



Quantify attack surface reduction.
Map to risk register.

GRC



Update compliance for “owned”
vs “rented” code risks.



The Future of Dependencies



We are moving from blind accumulation
to strategic selection.

**Stop importing the forest.
Plant the tree you need.**